



ACTAI • WEEK 04 • LECTURE 01

Apps That Remember

Databases made simple — with Google Sheets as your very first one.



Live build: a restaurant inventory system

THE PROBLEM

Right now, your apps forget



Add data to a vibe-coded app, hit refresh — and it's all gone. Nothing is saved.

Without a database

Data lives only in the page. Close it, lose it. No sharing between people or devices.



With a database

Data is saved for good. It survives refreshes, and everyone sees the same up-to-date info.

THE CONCEPT

What is a database, really?

Forget the scary word. A database is just a table — like a spreadsheet your app can read from and write to.



A table

one for each kind of thing (e.g. inventory items)



Columns = fields

the details you store (name, quantity, price...)



Rows = records

one row is one thing (one item, one customer)

If you can picture a spreadsheet, you already understand a database.

THE FOUR MOVES

Everything a database does: CRUD



Create

add a new row

"Add a new item"



Read

look at the rows

"View the stock list"



Update

change a row

"Change a quantity"



Delete

remove a row

"Remove an item"

Create, Read, Update, Delete — our inventory app will do all four.

Why start with Google Sheets



A Google Sheet is the friendliest database there is — perfect for learning and for real small apps.

- You can SEE your data as rows & columns
- Free, and you already have a Google account
- Easy to edit by hand when you need to
- Great up to a few thousand rows — plenty to start

DEMO LIVE: Open Google Sheets and show a blank sheet — 'this is our database'.

THE PLAN

Three pieces: Sheet → API → App



The Sheet

your data lives here



An API

a web link your app can call



The App

reads & writes the data

The middle piece (the API) sounds technical — but we'll create it in about two minutes, free.

STEP 1

Build the Sheet (our database)

Make a new Google Sheet. Row 1 = column names (the fields). Each row = one item (a record).

Item	Category	Quantity	Unit	ReorderLevel
Tomatoes	Vegetables	12	kg	5
Chicken breast	Meat	8	kg	10
Olive oil	Pantry	3	L	4
Mozzarella	Dairy	6	kg	5

Reorder level = the quantity at which we're running low and should order more. Our app will flag these in red.

STEP 2

Turn the Sheet into an API

In your Sheet: Extensions → Apps Script. Delete the sample, paste this, and Save. (You don't need to read it — just paste.)

APPS SCRIPT — paste this (it gives your sheet a read + add API)

```
const SHEET_ID = 'SHEET_ID_HERE';
const SHEET_NAME = 'Expenses';

function getSheet_() {
  const ss = SpreadsheetApp.openById(SHEET_ID);
  let sh = ss.getSheetByName(SHEET_NAME);
  if (!sh) {
    sh = ss.insertSheet(SHEET_NAME);
    sh.appendRow(['id', 'date', 'amount', 'category', 'note', 'createdAt']);
  }
  return sh;
}

function doGet() {
  const sh = getSheet_();
  const values = sh.getDataRange().getValues();
  const [header, ...rows] = values;
  const data = rows.map(r => Object.fromEntries(header.map((h, i) => [h, r[i]])));
  return ContentService.createTextOutput(JSON.stringify(data))
```


STEP 2, CONTINUED

Deploy it & copy the link

1. Click **Deploy** → **New deployment**
2. Set type to **Web app**
3. Execute as: **Me**
4. Who has access: **Anyone**
5. Deploy, then **Authorize** your account
6. Copy the **Web app URL** — that's your API!



The URL ends in **/exec**

Keep that link handy — your app will use it to read and add data.



Prefer zero code?

SheetDB.io or Sheety.co turn a Sheet into an API for you — quick signup, free tier.

STEP 3

Connect it in your app

Now build the app in any vibe-coding tool and tell it to use your URL. Same idea as last week's API key: your app just calls a web link.

TELL YOUR APP

Use this Google Sheet as the database: [PASTE YOUR /exec URL].

Load all items with a GET request to that URL.

Add a new item with a POST request to the same URL.

The sheet's columns are the fields.



GET = Read — fetch the rows to show them



POST = Add — send a new row to save

WATCH THIS

Live build: restaurant inventory

Build this in your vibe-coding tool, pasting your own /exec URL where shown.

BUILD PROMPT – paste into your tool

Build a restaurant inventory manager. Load items from this Google Sheet API with a GET request: [PASTE YOUR /exec URL]. Show all items in a clean table: Item, Category, Quantity, Unit, Reorder Level. Highlight a row in red when Quantity is at or below its Reorder Level (low stock). Add a form to add a new item that POSTs to the same URL. Modern restaurant theme with a header.

THEN TEST IT

Add 'Garlic, Vegetables, 2, kg, 5' – it saves to the Sheet and appears in the table. Open the Sheet: the new row is really there. Refresh the app: it's still there.

IT'S ALIVE

CRUD, in your inventory app



Create

Add 'Garlic' → new row in the Sheet



Read

The table loads every item from the Sheet



Update

Change a quantity → ask the app to add editing



Delete

Remove an item → a feature for next time

Our build does Create + Read today. Update + Delete are just two more prompts — try them live if time allows.

BUILD THESE TOO

More apps powered by a Sheet



Table reservations

Sheet: Name, Date, Time, Party size



Expense tracker

Sheet: Date, Category, Amount, Note



Event RSVP

Sheet: Name, Email, Attending, Guests



Feedback collector

Sheet: Name, Rating, Comment, Date



Simple CRM

Sheet: Name, Company, Email, Status



Class attendance

Sheet: Student, Date, Present

KNOW THE LIMITS

When a Sheet isn't enough



Best under a few thousand rows

It slows down with big data — fine for small apps.



No real logins yet

Anyone with the URL can read/write — auth comes next lecture.



Not for sensitive data

It's a shared sheet; keep secrets out of it.



Simple data only

Complex links between tables need a real database.

A Sheet is a brilliant START. Outgrow it and you graduate to a real database — that's next lecture.



LECTURE RECAP

Your apps remember now.

A database is just a table. A Google Sheet + a two-minute API = data that's saved for good. You built a real inventory system.

Homework • build one Sheet-powered app for your field. **Next** — real databases & logins.